From Data Discovery to Analytics-Ready Data: A Survey of Search, Preparation, and Integration in Data Lakes

Sumaiya Azad
University of Utah
Salt Lake City, Utah, USA
sumaiya.azad@utah.edu

Abstract

Organizations collect many kinds of data in data lakes. Before analysts can query those data or build dashboards from them, they must find relevant sources, clean and organize them, and combine information across sources.

I review 47 methods published from 2022 to 2025 and organize them into three phases: *Discovery & Search*, *Preparation & Wrangling*, and *Integration & Matching*. For each phase, I describe common tasks, system designs, and evaluation practices. I close with open challenges in end-to-end evaluation, keeping results up to date as data changes, and reliable use of language models for data tasks.

CCS Concepts

• **Information systems** → **Information integration**; **Data cleaning**; *Data exchange*; *Data mining*; *Data retrieval*.

Keywords

data lakes, dataset discovery, table search, join discovery, union discovery, data preparation, data wrangling, data cleaning, schema matching, entity resolution, data integration, analytics-ready data

1 Introduction

Data lakes [2, 14] store many kinds of data together—spreadsheet-like tables, semi-structured files such as JSON or XML, server logs, and fields pulled from documents—without requiring one fixed column layout, or *schema*, at load time. Loading data is easy, but using it later is hard. Before analysts can run queries, build dashboards, or train models, they must (1) find the right data, (2) clean and organize it, and (3) combine it from multiple sources.

Scope and terminology. *Discovery* means finding relevant tables or columns, including *join* search [14] (tables sharing a key column) and *union* search [2] (tables with compatible columns). *Preparation* means cleaning errors, transforming values, and organizing raw data into usable tables or reusable workflows. *Integration* means lining up columns across sources and producing a merged output; *entity resolution (ER)* is the special case of deciding whether two records describe the same real-world thing. I use *analytics-ready data* to mean data that is clean and structured enough for reports, statistics, or machine learning.

Two trends make this pipeline harder: (1) modern lakes are large and weakly documented; (2) embedding-based and language-model-based methods now appear in discovery [24], preparation [9], and integration [32], raising concerns about cost, reliability, and privacy.

This survey follows the full path from raw lake data to analytics-ready outputs in three phases: (1) *Discovery & Search* finds relevant tables and ranks candidates that could be joined or unioned; (2) *Preparation & Wrangling* cleans, restructures, and transforms selected data into usable tables or reusable automated workflows; and (3) *Integration & Matching* lines up columns and records across sources to produce a single merged dataset. I describe my six-step process for selecting and organizing methods in Section 3.

My main focus is the connection among these three phases. Instead of treating discovery, preparation, and integration as separate tasks, I study them as one pipeline where each stage feeds the next.

Contributions.

- I review methods from three top database venues (SIGMOD, PVLDB, and ICDE) using a clear process for selecting and categorizing them (Section 3).
- I propose a taxonomy that groups methods by what they produce—ranked candidate lists, cleaned tables, automated workflows, column matches, or merged datasets—spanning discovery, preparation, and integration (Section 4.2).
- I summarize the tasks, system designs, and evaluation practices in each phase, including trade-offs between effectiveness, cost, and the need for human oversight (Section 5.3.4).
- I identify open problems, including pipeline-level evaluation, keeping results up to date as data changes, reliable use of language models, fairness and privacy constraints, and the need for shared test collections (Section 6).

2 Related Work

Earlier surveys cover *discovery* in data lakes in depth, including keyword search, join discovery, and union discovery [14], as well as indexing and benchmarking [2]. These surveys clearly define the main discovery tasks and compare systems in detail.

Other work studies individual tasks such as *data cleaning* [35], *schema matching* [32], and *entity resolution* [20]. I instead focus on recent work across the three phases, which lets me capture language-model-based methods, newer system designs, and emerging concerns such as cost, reliability, fairness, and privacy.

3 Methodology

This section explains how I found, selected, and organized the methods in this survey, with enough detail to reproduce the search and check inclusion decisions.

Step 1: Research questions. For each method I ask four questions: (i) which phase does it target (discovery, preparation, or integration)? (ii) what does it mainly produce (for example, a ranked list, a cleaned table, or a merged dataset)? (iii) what technique does it use? and (iv) how is it evaluated (which datasets, which metrics, and whether cost is reported)?

Step 2: Search terms. I search using the query `"data lake" AND (S1 OR S2 OR S3 OR S4)`, where S1 through S4 are groups of task-related keywords listed in Table 1.

Table 1: Search term groups used for queries over data lakes.

ID	Query family (examples of task keywords)
S1	discovery, search, table search, dataset retrieval, exploration
S2	joinable, unionable, overlap, join discovery, union discovery
S3	wrangling, preparation, cleaning, transform, pipeline synthesis
S4	schema matching, entity resolution, record linkage, data integration, LLM

Step 3: Sources and retrieval. I scan proceedings of three top database venues—SIGMOD, PVLDB, and ICDE—by running title and abstract queries with the keyword groups S1–S4. The final pool contains 49 papers: 15 from SIGMOD, 22 from PVLDB, and 12 from ICDE. Among them, 47 are method papers. The remaining 2 are discovery survey/tutorial papers that I keep for background discussion but do not count as methods.

Step 4: Inclusion and exclusion. I include method papers that study a data lake or lakehouse setting and that propose a concrete technique for discovery, preparation, or integration. I exclude papers that do not target a lake setting. I also exclude survey and tutorial papers from the method count, while still discussing them as background where they help contextualize the discovery literature.

Step 5: Coding rule. Each method is assigned to exactly one phase based on its *primary output*—the main thing it produces. Specifically: a method that outputs a ranked list of candidate tables goes into Discovery; a method that outputs a cleaned table, transformed data, or a reusable processing workflow goes into Preparation; and a method that outputs column matches, record-level entity matches, or a merged dataset goes into Integration. This rule prevents double-counting and makes the taxonomy clear.

Step 6: Synthesis. I group the 47 selected method papers by the taxonomy defined in Section 4.2. The distribution across phases is 17 papers in Discovery and Search, 13 in Preparation and Wrangling, and 17 in Integration and Matching. The later literature-review sections describe techniques and evaluation practices within each category, while also noting the discovery survey/tutorial papers separately where they provide useful context.

4 Taxonomy Overview

I organize the selected methods into a two-level taxonomy. The top level has three *classes*, one for each phase of the pipeline: Discovery, Preparation, and Integration. Within each class, I define *categories* that capture common method patterns.

4.1 Classification Summary

Figure 1 shows the three classes and their categories. Together, they reflect the common path from raw lake contents to analytics-ready outputs.

4.2 Classes and Their Categories

Below, I define each class by its primary output and recurring tasks. The full category breakdown is shown in Figure 1.

4.2.1 Discovery and Search.

Output. A ranked list of candidate data sources (tables, columns, or derived views) for the analyst to explore, join, or union.

End-to-end systems combine background work such as profiling tables and keeping indexes up to date with real-time search and interactive exploration [7, 13, 24]. Representation methods learn compact vector embeddings [15, 38] or statistical sketches [26] for columns and tables, so that search can rely on meaning rather than exact text matches, even when metadata is missing or inconsistent. Join search, union search, and overlap estimation find candidate tables that share data with a given seed table, then score how strongly each candidate is related. These are discovery-time steps: they identify candidates but do not produce the final merged table. Corpora such as GitTables and benchmarks such as LakeBench [10] support fair comparison and model training at lake scale.

4.2.2 Preparation and Wrangling.

Output. Cleaned or transformed tables, well-structured relational views, and reusable automated workflows that can run again when new data arrives.

Cleaning methods fix errors and reduce data quality problems [35]. Language-model-based curation methods [9, 34] and transform-reuse systems [1] build reusable cleaning workflows—for example, a pipeline that detects and corrects misspelled city names. Pipeline synthesis methods [8, 17] search for the best sequence of processing steps (such as removing duplicates, then standardizing dates, then adding a computed column) that improve results for a downstream task like training a classifier. Closely related automation methods augment features [30] or assemble BI models from input tables [31]. Structuring methods convert messy inputs (such as nested JSON files or inconsistent spreadsheets) into clean relational tables [4, 29, 47]. Ingestion debugging methods help diagnose errors that occur when data is first loaded into the lake [6].

4.2.3 Integration and Matching.

Output. Column matches, entity matches or clusters, and merged datasets.

Schema and table integration methods line up columns across multiple tables and apply explicit merge rules that determine how partial facts from different sources are combined [27, 28, 32, 49]. ER workflows combine *blocking*, *matching*, and *clustering*. These workflows must balance labeling cost against accuracy [19, 20, 33, 41, 44]. Evaluation work studies speed-versus-quality trade-offs in filtering [36], subgroup fairness [39], and low-cost feature construction [40, 45]. Other methods produce shared evidence—such as approximate foreign-key relationships [23], shared embeddings [42], and selectively retrieved external data [5, 46]—that can help multiple integration tasks at once.

5 Literature Review

5.1 Discovery and Search

The reviewed discovery literature addresses retrieval in large lakes with weak metadata and incomplete documentation. It includes 17 counted method papers plus 2 survey/tutorial papers that I discuss for context.

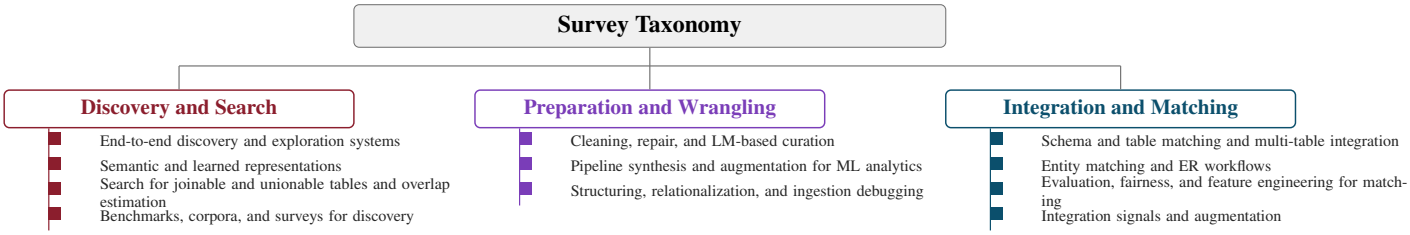


Figure 1: Taxonomy of data lake research from discovery and search to analytics-ready outputs via preparation and integration.

The literature shows four recurring patterns. First, systems are moving from single search features to end-to-end stacks that handle indexing, search, and user interaction together. Second, retrieval quality increasingly depends on learned embeddings rather than metadata alone. Third, join search, union search, and overlap estimation serve as building blocks for later integration. Fourth, shared corpora and benchmarks are essential for fair comparison.

5.1.1 End-to-end Discovery and Exploration Systems. End-to-end discovery systems return ranked tables, derived views, or dataset augmentations. A common architecture has three parts: (i) offline enrichment (profiling, indexing); (ii) online retrieval (keyword, join, union, and view search); and (iii) interactive exploration.

CHORUS uses a large language model (a "foundation model") to produce shared understanding of each table—for example, detecting what category a table belongs to, what type each column is, and which columns could be used to join tables together—and then reuses those annotations for indexing, filtering, and ranking. Because one model drives several tasks, signals can be reused efficiently, but prompt sensitivity, non-determinism, and cost can also affect multiple decisions at once. BLEND makes the discovery pipeline explicit and tunable by introducing *discovery plans*—multi-step workflows written in a declarative style (the user states *what* to find, not *how*)—that mix retrieval, composition, and ranking steps while connecting offline indexes to real-time execution. LakeCompass packages the same general idea as a single operational stack for storage, indexing, keyword search, join and union search, and interactive exploration.

Other systems specialize the user-facing side of this process. METAM [16] lets downstream task utility steer ranking, so candidate datasets are judged by whether they improve the target analysis rather than only by similarity or joinability. Selecting Sub-tables [3] makes large results easier to inspect by surfacing compact, informative previews, while Ver [21] handles ambiguity in *view discovery* by exploring multiple join structures, pruning redundant options, and asking targeted questions to narrow down the intended merged view.

5.1.2 Semantic and Learned Representations for Discovery. Because meaning is rarely written down explicitly in data lakes, many methods learn compact embeddings for columns and tables so that search can compare meaning rather than exact text.

CMDL [12] extends discovery beyond tables by embedding columns and unstructured documents into a shared vector space, using weak supervision to create training signal when labeled cross-modal examples are scarce. Starmie learns *contextualized* column embeddings in which each column is interpreted through its table context, making semantic union search more robust than matching on isolated column text alone. LIFTus addresses heterogeneous columns such as numeric IDs and patterned strings by encoding each column through multiple *aspects* and fusing them into a single retrieval representation, while TabSketchFM reduces token and cost overhead by compressing each table into a compact statistical sketch that a transformer can read directly.

5.1.3 Join and Union Search, and Overlap Estimation. Join search, union search, and overlap estimation are tightly connected building blocks for *table enrichment*—the process of adding useful rows or columns to a starting table. Given a *seed table* (the analyst's starting point), systems retrieve a small set of candidates that can be joined (linked by a shared key) or unioned (stacked because they have compatible columns), then score how strongly each candidate relates to the seed. This is still discovery: the system identifies candidates but does not yet produce a final merged table.

DeepJoin [11] treats joinability as a learned retrieval problem by encoding columns with a pre-trained language model and retrieving top-k join partners through approximate nearest-neighbor indexing, making search more robust to spelling and formatting variation. SANTOS [25] focuses on unionability by modeling semantic relationships across columns rather than relying only on surface overlap, while Nested/Joined Lake Search [48] extends the candidate space to *join-nest* results built dynamically from multiple tables. Armadillo [37] abstracts across join, union, and deduplication by learning graph-based embeddings whose similarity approximates table overlap at scale.

5.1.4 Benchmarks, Corpora, and Surveys for Discovery. Discovery at lake scale depends on shared resources—corpora for training, benchmarks for fair comparison, and indexing techniques for efficient search.

Rottnest [43] addresses the operational side of discovery by proposing a lakehouse indexing protocol that manages index-building and update overhead in evolving storage. GitTables [22] supplies a large relational-style corpus with semantic-type annotations for representation learning and table-understanding tasks, and LakeBench provides labeled workloads for join and union discovery so systems can be compared on both quality and runtime. The two survey papers complement these resources from different angles: the Data Discovery Survey emphasizes operational concerns such as mismatched evaluation setups and weak reporting of computational cost, while the Table Discovery Survey consolidates methods and open problems around robustness and efficiency.

Once candidate data sources are surfaced, the next phase focuses on transforming these often-messy results into clean, well-structured inputs that can be used for analytics and downstream integration.

5.2 Preparation and Wrangling

The reviewed preparation literature emphasizes turning discovered candidates into stable inputs for analysis. The main idea is *repeatable automated workflows* instead of one-time manual fixes. Because discovery often retrieves messy data and integration assumes reasonably well-structured inputs, preparation directly affects how well later matching works. If preparation fails to standardize date formats, for example, integration may incorrectly treat matching dates as different values—an impact easy to miss when evaluation only reports local metrics.

5.2.1 Cleaning, Repair, and LM-based Curation. Data lakes make classic data-quality problems worse: errors are frequent and come in many forms, and the context needed to understand dirty values is often unavailable. Methods in this category focus on two goals: (i) *deployable repair* that actually reduces the number of errors in a table, and (ii) *reusable curation workflows* that turn cleaning logic into automated, controllable steps.

Different preparation methods optimize different goals—consistency, net error reduction, or downstream model accuracy—and the automatic data repair study [35] shows why mixing these goals without stating them clearly can be misleading.

Automatic Data Repair argues that repair methods should be evaluated by whether they reduce the total number of errors, not just by how many cells they change, and benchmarks methods across datasets and error types while proposing safeguards against *incorrect repairs*. Foundation Models for Data Tasks [34] recasts traditional preparation tasks as prompts to large language models, showing broad task coverage but also surfacing privacy concerns and failures on domain-specific conventions. Lingua Manga turns that idea into reusable curation pipelines through modular templates and a small scripting language that reduces expensive model calls and supports rerunnable workflows as new lake data arrives, while DataRinse reaches similar reuse goals without language models by mining semantic transformation patterns from existing preparation code and presenting them as coherent human-in-the-loop recommendations.

5.2.2 Pipeline Synthesis and Augmentation for ML Analytics. Pipeline synthesis treats data preparation as a search problem: given raw tables and a goal (such as training an accurate classifier), the system automatically finds the best sequence of processing steps—cleaning, transforming, and adding features—that improves the final result within a given budget.

CtxPipe addresses the huge search space of possible processing sequences by using dataset characteristics and task feedback to prioritize transformations that are more likely to improve the downstream model. HAIPipe [8] keeps that search grounded in expert intent by starting from a human-written pipeline and augmenting it automatically, while Featpilot [30] treats augmentation as join-path search over other lake tables so that useful features can be added through multi-hop joins. Auto-BI [31] pushes the same idea toward business-intelligence outputs by discovering the *join model* needed to connect multiple tables for dashboards and reports.

5.2.3 Structuring, Relationalization, and Ingestion Debugging. This category covers two related problems: (1) converting messy lake inputs (such as nested JSON files, HTML tables, or unstructured documents) into clean, queryable relational tables (standard row-and-column format), and (2) detecting and fixing errors that occur during *ingestion*—the process of loading data into the lake.

Evaporate [4] builds queryable tables from unstructured documents by either extracting values with a language model or synthesizing extraction code, and can combine multiple synthesized extractors with weak supervision to improve coverage. Auto-Tables [29] tackles irregular spreadsheet layouts by synthesizing multi-step relationalization programs without input–output examples, while ReCG [47] induces usable schemas from semi-structured collections by clustering records, generalizing clusters into schema fragments, and merging them into a compact global schema. On the debugging side, CM Explorer isolates ingestion failures through conditional data tests over targeted subsets rather than only global statistics, and XDBC [18] treats cross-system data movement itself as a configurable transfer pipeline whose stages can be auto-tuned for throughput.

5.3 Integration and Matching

The reviewed integration literature organizes around four concerns: (i) schema-level alignment and merge rules, (ii) entity resolution as a multi-step workflow, (iii) fairness-aware evaluation, and (iv) reusable integration signals such as approximate foreign-key relationships or learned embeddings. Here, an approximate foreign-key relationship is a noisy join hint: values in one column often correspond to values in another even when formatting differences or typos prevent exact matches. A broader trend is that integration outputs increasingly feed back into discovery and preparation as reusable evidence.

5.3.1 Schema and Table Matching and Integration. Schema matching and table integration form the core of the alignment process. Given a set of candidate tables found during discovery, systems must determine which columns correspond to each other, then apply merge rules to combine everything into one analysis-ready table. This is different from join or union search in the discovery phase: here the system actually performs the merge and produces a final output.

End-to-end integration pipelines. DIALITE treats integration as a complete workflow: first *discover* relevant tables, then *align and integrate* them by matching columns and merging rows, and finally *analyze* the merged table. Different discovery methods can feed into the same alignment stage, and intermediate outputs (candidate set, column alignments, merged table) can be inspected at each step.

Integration semantics and operators. ALITE argues that integration quality depends on clear merge rules, often called *integration semantics*, not just pairwise joins. For example, when one table has a customer's phone number but not their email, and another table has the email but not the phone number, the merge rule should combine both pieces of information into one row. ALITE first performs global column matching by assigning shared IDs to matched columns, then applies *Full Disjunction*—a merge strategy designed for tables with many missing values and partial overlaps. It breaks this into reusable operators: outer union, subsumption, and complementation.

The core idea, also central in DIALITE, is to connect partial facts across sources rather than just running local joins. Discovery finds

the tables; the harder step is deciding exactly how to combine them so the final output is complete and useful.

LM-powered schema matching. Magneto uses a two-stage design: a smaller, cheaper language model first proposes and ranks candidate column matches, and then a larger, more powerful model validates the hard cases for higher precision. PLM Schema Matching targets enterprise settings where access to actual data is limited (for privacy reasons) and the target schema is large and domain-specific. It adapts pre-trained language models with minimal labeled examples to reduce the effort needed from human experts.

5.3.2 Entity Matching and ER Workflows.

This literature treats ER as a workflow rather than a single classifier.

Progressive and intent-aware matching. Progressive EM studies pay-as-you-go ER: when time or budget is limited, the system prioritizes the most valuable comparisons first. It breaks the process into filtering, weighting, scheduling, and matching steps. FlexER shows that what counts as a "match" can depend on downstream intent: the same two product listings may be treated as equivalent for one task and distinct for another. It models those multiple valid decisions as multi-label classification.

Lowering supervision cost and unifying models. Obtaining labeled training data for EM is expensive. WeakSup GT Inference addresses this by combining noisy weak-supervision signals with a label model that enforces global constraints such as transitivity. Battleship targets low-resource settings with active learning that strategically selects the most informative record pairs for a human to label. Unicorn unifies multiple matching tasks into a single shared model with task-specific output heads.

5.3.3 Evaluation, Fairness, and Feature Engineering for Matching.

Recent EM work treats matching as a configurable pipeline where blocking strategies, fairness across demographic groups, and feature engineering all matter.

Benchmarking filtering under quality–runtime targets. Filtering Benchmarks compares different blocking strategies (for example, rule-based blocking versus approximate nearest-neighbor retrieval) for candidate-pair generation. It shows that fair comparisons require careful tuning of many interacting parameters, and reports effectiveness alongside running time and scalability.

Fairness-aware evaluation for EM. Fairness Lens audits EM methods across demographic subgroups—for example, checking whether matching accuracy differs between common names and uncommon names, or between majority and minority groups. It shows that overall accuracy can look good while hiding significant disparities for specific subgroups, and advocates for fairness-sensitive metrics that go beyond population-level averages.

Practical feature selection and attribute enrichment. DescSim Rules mines interpretable similarity rules by analyzing column descriptions and value distributions, allowing expensive model retraining to be done offline. Relation Enrichment improves matching by adding a small set of carefully chosen auxiliary attributes from related tables—for example, adding a customer's city to help disambiguate customers with the same name.

5.3.4 Integration Signals and Augmentation.

This category covers methods that output approximate foreign-key relationships, shared embeddings, or targeted augmentation results rather than a final merged table.

Relaxed dependency signals for noisy join discovery. SIND introduces *similarity inclusion dependencies* (sINDs). In a clean database, a foreign-key relationship means every value in one column appears exactly in another. In noisy data lakes, exact containment rarely holds, so sINDs allow sufficiently similar values instead. That makes them useful as approximate foreign-key hints for join discovery and integration.

Reusable representation signals across tasks. Sudowoodo pretrains similarity-aware embeddings using contrastive learning on unlabeled data. These embeddings can then be reused across multiple integration tasks, reducing the need for labeled training data.

On-demand external enrichment via progressive querying. Entity Augmentation studies how to enrich lake entities using external sources that are only accessible through keyword-search interfaces (such as web APIs). It proposes a progressive, feedback-driven querying strategy: submit initial queries, check which results are useful, and refine subsequent queries based on what was found.

Analysis-driven augmentation for causal validity. Causal DI addresses a specific analytics need: ensuring that integrated data supports valid causal analysis. It looks for missing *confounders*, meaning variables that influence both the treatment and the outcome, retrieves them from external sources, and then builds a causal graph to decide which variables should be integrated.

6 Discussions and Challenges

This section presents the main lessons from the survey and the open problems that remain.

6.1 Phases Are Connected but Evaluation Is Isolated

Most methods report metrics for only one phase, which can hide downstream problems. A discovery method with high precision may still retrieve tables that are hard to integrate. A practical lesson is to report stage-level results, maintenance cost, and at least one end-to-end outcome together.

Methods in different phases also optimize different goals—ranking quality, format consistency, or match accuracy—and without naming these differences, cross-method comparisons can look cleaner than they actually are.

6.2 Ambiguity and User Intent

In data lakes, metadata is often missing or vague, and the same query can have many plausible answers. For example, searching for "customer data" might return sales records, support tickets, or demographic profiles—all valid depending on the analyst's goal. Discovery systems address this by supporting interactive refinement rather than assuming a single correct answer. On the integration side, the same pair of records can be considered a "match" or "non-match"

depending on the downstream task, as FlexER illustrates with intent-aware entity resolution. This challenges the single-ground-truth assumption built into many benchmarks. An important open problem is *intent-conditioned evaluation*: explicitly varying the analyst's goal and reporting how robust each method is across different intents and domains.

6.3 Scalability, Cost, and Maintenance

At the scale of real data lakes (thousands to millions of tables), major costs come from infrastructure: generating embeddings for every column, building and maintaining search indexes, and keeping everything up to date as new data arrives. End-to-end systems, benchmarks, and operations surveys all show why cost should be reported alongside effectiveness. Preparation and integration face similar pressures: pipeline synthesis requires repeated execution, progressive ER uses filtering and scheduling to control cost on a pay-as-you-go basis, and filtering benchmarks quantify the trade-off. Most current evaluations assume that the data does not change over time, which hides the real challenge of keeping results up to date as lake contents evolve. A key research direction is incremental maintenance—updating indexes, embeddings, and validation tests efficiently when new data arrives, rather than rebuilding everything from scratch.

6.4 Preparation Needs Safety Checks

Preparation can improve downstream quality, but improvements are not guaranteed. A repair system might over-edit or introduce plausible but incorrect values. Evaluation should therefore prioritize net error reduction and avoid changes that introduce more errors than they fix. Language-model-based curation adds further risks: models can hallucinate, give different answers on repeated runs, and expose private data if data is sent to an external API. Structuring methods also show that converting messy lake inputs into clean tables often requires multiple transformation steps, not just one. In practice, systems need lightweight verification hooks such as automated tests, constraint checks, and sample-based audits that can trigger a rollback or request human review when confidence is low.

6.5 Integration Design Choices

Integration in data lakes is not as simple as "match columns, then join." Real-world tables overlap only partially and have many missing values. Merge rules must therefore preserve partial facts while combining as much evidence as possible. Full Disjunction-style approaches show that careful merge operators can significantly outperform naive pairwise joins. At the entity level, workflow-oriented ER shows that the choice of blocking and scheduling strategy and supervision strategy often matters more for end-to-end performance and cost than the choice of matching model alone. Unified matching models address part of this design space. Language-model-based matchers can reduce the need for labeled data and handle diverse formats, but they raise concerns about the reliability of model confidence scores, latency, and privacy. A key open issue is choosing merge rules and workflow steps that align with the analyst's actual goal, and evaluating those trade-offs directly.

6.6 Fairness, Privacy, and Governance

As lake pipelines become more automated, responsible operation becomes part of system design, not an afterthought. Fairness-aware evaluation shows that overall accuracy can mask unequal error rates across groups—for example, an entity-matching system may work well for common names but poorly for uncommon ones. Language-model-based data tasks also raise privacy risks: converting data into text prompts and sending them to an external API can leak sensitive information or violate organizational policies. Governance rules—organizational policies about which tables can be joined, which attributes can be exposed, and which external sources can be used—should be built into the pipeline as planning constraints, not checked only after the fact. Open problems include auditing fairness at every stage (not just the final matcher), developing privacy-preserving indexing and language-model techniques, and building integration planners that respect organizational policies by design.

6.7 Benchmark Gaps and Shared Test Collections

Shared corpora and benchmarks have improved comparability within each phase: GitTables and LakeBench standardize parts of discovery evaluation, and ER benchmarks support workflow comparison. Learned embeddings and profiling signals are also reused across tasks. However, few resources cover the full pipeline—discovery through preparation through integration—under realistic conditions such as data updates, changing analyst intent, and explicit cost reporting. A concrete next step is a pipeline test collection that combines workloads from all three phases, provides ground truth at each stage, and tracks computational cost.

Such a test collection is especially useful because failures often occur at the boundaries between phases—for example, discovery returning structurally incompatible tables, or preparation removing values needed for later matching.

6.8 Threats to Validity

This study has limits. I searched three venues (SIGMOD, PVLDB, ICDE), so methods published elsewhere may be missing. Some methods could fit more than one category, so any taxonomy compresses real overlap. The chosen time window keeps the survey focused on recent work, but it excludes methods outside that window. Methods across phases optimize different goals such as ranking quality, consistency, and match accuracy, so side-by-side comparison is inherently imperfect. Limited code and data release makes full reproducibility difficult. I keep phase-specific metrics separate and note where reproducibility is weak.

7 Conclusion

I reviewed recent work across discovery, preparation, and integration. Discovery increasingly combines learned representations with scalable indexing and retrieval; preparation increasingly produces repeatable automated workflows but still needs safety checks; and integration increasingly depends on workflow design and analyst intent rather than a single matching score. A natural next step is stronger end-to-end evaluation under realistic conditions including data updates, ambiguous intent, and explicit cost measurement.

References

[1] Ibrahim Abdelaziz, Julian Dolby, Udayan Khurana, Horst Samulowitz, and Kavitha Srinivas. 2023. DataRinse: Semantic Transforms for Data Preparation Based on Code Mining. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 4090–4093. doi:10.14778/3611540.3611628

[2] Ziawasch Abedjan, Mahdi Esmailoghli, and Sainyam Galhotra. 2025. Data Discovery in Data Lakes: Operations, Indexes, Systems. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5455–5459. doi:10.14778/3750601.3750694

[3] Yael Amsterdamer, Susan B. Davidson, Tova Milo, Kathy Razmadze, and Amit Somech. 2023. Selecting Sub-tables for Data Exploration. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2496–2509. doi:10.1109/ICDE55515.2023.00192

[4] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 92–105. doi:10.14778/3626292.3626294

[5] Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. 2023. Effective Entity Augmentation by Querying External Data Sources. *Proc. VLDB Endow.* 16, 11 (July 2023), 3404–3417. doi:10.14778/3611479.3611535

[6] Niels Bylois, Frank Neven, and Stijn Vansummeren. 2023. CM-Explorer: Dissecting Data Ingestion Problems. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3958–3961. doi:10.14778/3611540.3611595

[7] Chengliang Chai, Yuhao Deng, Yutong Zhan, Ziqi Cao, Yuanfang Zhang, Lei Cao, Yuping Wang, Zhiwei Zhang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4381–4384. doi:10.14778/3685800.3685880

[8] Sibei Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. doi:10.1145/3588945

[9] Zui Chen, Lei Cao, and Sam Madden. 2023. Lingua Manga: A Generic Large Language Model Centric System for Data Curation. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 4074–4077. doi:10.14778/3611540.3611624

[10] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.* 17, 8 (April 2024), 1925–1938. doi:10.14778/3659437.3659448

[11] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.* 16, 10 (June 2023), 2458–2470. doi:10.14778/3603581.3603587

[12] Mohamed Y. Eltabakh, Mayuresh Kunjir, Ahmed K. Elmagarmid, and Mohammad Shahmeer Ahmad. 2023. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. *Proc. VLDB Endow.* 16, 11 (July 2023), 3377–3390. doi:10.14778/3611479.3611533

[13] Mahdi Esmailoghli, Christoph Schnell, Renée J. Miller, and Ziawasch Abedjan. 2025. BLEND: A Unified Data Discovery System. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 737–750. doi:10.1109/ICDE65448.2025.00061

[14] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data* (Seattle, WA, USA) *(SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 69–75. doi:10.1145/3555041.3589409

[15] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning. *Proc. VLDB Endow.* 16, 7 (March 2023), 1726–1739. doi:10.14778/3587136.3587146

[16] Sainyam Galhotra, Yue Gong, and Raul Castro Fernandez. 2023. Metam: Goal-Oriented Data Discovery. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2780–2793. doi:10.1109/ICDE55515.2023.00213

[17] Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6, Article 231 (Dec. 2024), 27 pages. doi:10.1145/3698831

[18] Haralampos Gavriilidis, Kaustubh Beedkar, Matthias Boehm, and Volker Markl. 2025. Fast and Scalable Data Transfer Across Data Systems. *Proc. ACM Manag. Data* 3, 3, Article 157 (June 2025), 28 pages. doi:10.1145/3725294

[19] Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4, Article 224 (Dec. 2023), 25 pages. doi:10.1145/3626711

[20] Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. FlexER: Flexible Entity Resolution for Multiple Intents. *Proc. ACM Manag. Data* 1, 1, Article 42 (May 2023), 27 pages. doi:10.1145/3588722

[21] Yue Gong, Zhiru Zhu, Sainyam Galhotra, and Raul Castro Fernandez. 2023. Ver: View Discovery in the Wild. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 503–516. doi:10.1109/ICDE55515.2023.00045

[22] Madelon Hulsebos, Çagatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1, Article 30 (May 2023), 17 pages. doi:10.1145/3588710

[23] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1, Article 75 (May 2023), 24 pages. doi:10.1145/3588929

[24] Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. 2024. Chorus: Foundation Models for Unified Data Discovery and Exploration. *Proc. VLDB Endow.* 17, 8 (April 2024), 2104–2114. doi:10.14778/3659437.3659461

[25] Aamod Khatiwada, Grace Fan, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1, Article 9 (May 2023), 25 pages. doi:10.1145/3588689

[26] Aamod Khatiwada, Harsha Kokel, Ibrahim Abdelaziz, Subhajit Chaudhury, Julian Dolby, Oktie Hassanzadeh, Zhenhan Huang, Tejaswini Pedapati, Horst Samulowitz, and Kavitha Srinivas. 2025. TabSketchFM: Sketch-Based Tabular Representation Learning for Data Discovery Over Data Lakes. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1523–1536. doi:10.1109/ICDE65448.2025.00118

[27] Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. 2022. Integrating Data Lake Tables. *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 932–945. doi:10.14778/3574245.3574274

[28] Aamod Khatiwada, Roee Shraga, and Renée J. Miller. 2023. DIALITE: Discover, Align and Integrate Open Data Tables. In *Companion of the 2023 International Conference on Management of Data* (Seattle, WA, USA) *(SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 187–190. doi:10.1145/3555041.3589732

[29] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. 2024. Auto-Tables: Relationalize Tables without Using Examples. *SIGMOD Rec.* 53, 1 (May 2024), 76–85. doi:10.1145/3665252.3665269

[30] Jiaming Liang, Chuan Lei, Xiao Qin, Jiani Zhang, Asterios Katsifodimos, Christos Faloutsos, and Huzefa Rangwala. 2025. Featpilot: Automatic Feature Augmentation on Tabular Data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2148–2160. doi:10.1109/ICDE65448.2025.00163

[31] Yiming Lin, Yeye He, and Surajit Chaudhuri. 2023. Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph. *Proc. VLDB Endow.* 16, 10 (June 2023), 2578–2590. doi:10.14778/3603581.3603596

[32] Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. 2025. Magneto: Combining Small and Large Language Models for Schema Matching. *Proc. VLDB Endow.* 18, 8 (April 2025), 2681–2694. doi:10.14778/3742728.3742757

[33] Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. 2025. Progressive Entity Matching: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 1, Article 65 (Feb. 2025), 25 pages. doi:10.1145/3709715

[34] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 738–746. doi:10.14778/3574245.3574258

[35] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proc. VLDB Endow.* 17, 10 (June 2024), 2617–2630. doi:10.14778/3675034.3675051

[36] George Papadakis, Marco Fisichella, Franziska Schoger, George Mandilaras, Nikolaus Augsten, and Wolfgang Nejdl. 2023. Benchmarking Filtering Techniques for Entity Resolution. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 653–666. doi:10.1109/ICDE55515.2023.00389

[37] Francesco Pugnaloni, Luca Zecchini, Matteo Paganelli, Matteo Lissandrini, Felix Naumann, and Giovanni Simonini. 2025. Table Overlap Estimation through Graph Embeddings. *Proc. ACM Manag. Data* 3, 3, Article 228 (June 2025), 25 pages. doi:10.1145/3725365

[38] Ermu Qiu, Jun Gao, Yaofeng Tu, and Jingru Yang. 2025. LIFTus: An Adaptive Multi-Aspect Column Representation Learning for Table Union Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2174–2187. doi:10.1109/ICDE65448.2025.00165

[39] Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. 2023. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. *Proc. VLDB Endow.* 16, 11 (July 2023), 3279–3292. doi:10.14778/3611479.3611525

[40] Yafeng Tang, Zheng Liang, Hongzhi Wang, Xiaoou Ding, Tianyu Mu, and Huan Hu. 2025. Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1–15. doi:10.1109/ICDE65448.2025.00008

[41] Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1, Article 84 (May 2023), 26 pages. doi:10.1145/3588938

[42] Runhui Wang, Yuliang Li, and Jin Wang. 2023. Sudowoodo: Contrastive Self-supervised Learning for Multi-purpose Data Integration and Preparation. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1502–1515. doi:10.1109/ICDE55515.2023.00391

[43] Ziheng Wang, Sasha Krassovsky, Conor Kennedy, Alex Aiken, Weston Pace, Rain Jiang, Huayi Zhang, Chenyu Jiang, and Wei Xu. 2025. Rottnest: Indexing Data Lakes for Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1814–1827. doi:10.1109/ICDE65448.2025.00139

[44] Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. doi:10.1145/3588712

[45] Mengyi Yan, Wenfei Fan, Yaoshu Wang, and Min Xie. 2024. Enriching Relations with Additional Attributes for ER. *Proc. VLDB Endow.* 17, 11 (July 2024), 3109–3123. doi:10.14778/3681954.3681987

[46] Brit Youngmann, Michael Cafarella, Babak Salimi, and Anna Zeng. 2023. Causal Data Integration. *Proc. VLDB Endow.* 16, 10 (June 2023), 2659–2665. doi:10.14778/3603581.3603602

[47] Joohyung Yun, Byungchul Tak, and Wook-Shin Han. 2024. ReCG: Bottom-up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. *Proc. VLDB Endow.* 17, 11 (July 2024), 3538–3550. doi:10.14778/3681954.3682019

[48] Yi Zhang, Peter Baile Chen, and Zachary G. Ives. 2024. Searching Data Lakes for Nested and Joined Data. *Proc. VLDB Endow.* 17, 11 (July 2024), 3346–3359. doi:10.14778/3681954.3682005

[49] Yunjia Zhang, Avrilia Floratou, Joyce Cahoon, Subru Krishnan, Andreas C. Müller, Dalitso Banda, Fotis Psallidas, and Jignesh M. Patel. 2023. Schema Matching using Pre-Trained Language Models. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1558–1571. doi:10.1109/ICDE55515.2023.00123